Lecture 19: BERT variants

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College

Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Identify key limitations of the original BERT training procedure
2. Explain how RoBERTa, ALBERT, DistilBERT, ELECTRA, and DeBERTa each address different limitations
3. Compare parameter efficiency, training efficiency, and inference speed across variants
4. Select the appropriate variant for a given task and resource constraint
5. Argue whether encoder models remain relevant in the era of GPT-4

What could be improved?

BERT's key limitations (from Lecture 18)

Training procedure: NSP may hurt performance; static masking reuses the same masks every epoch; only 15% of tokens provide training signal (recall the 80/10/10 MLM procedure).

Scale: Trained on only 3.3B words with 100K steps — modern datasets are 100× larger (and often train for much longer).

Efficiency: 110M parameters are all active for every input. Large memory footprint for deployment.

Companion notebook

📓 Companion Notebook — try different BERT variants hands-on

RoBERTa: robustly optimized BERT

Key idea: better training = better performance

RoBERTa (Liu et al., 2019) keeps BERT's architecture but fixes the training recipe:

1. **Remove NSP** — Next Sentence Prediction hurt performance. Use only MLM with full sentences.
2. **Dynamic masking** — Generate a new masking pattern every time a sequence is seen, instead of reusing the same mask.
3. **Larger batches, more data** — Batch size 8K sequences (vs BERT's 256), 160GB text (vs 16GB), 500K steps (vs 100K).
4. **Longer sequences** — Train on longer contiguous text for better long-range understanding.

Takeaway

Training procedure matters as much as architecture. RoBERTa shows that BERT was significantly **undertrained**.

Dynamic masking

RoBERTa's key innovation

BERT used **static masking** — the same mask positions every epoch, risking memorization. RoBERTa generates **new masks on-the-fly** during training:

- Epoch 1: "My [MASK] is cute"
- Epoch 2: "My dog [MASK] cute"
- Epoch 3: "My dog is [MASK]"

More diverse training signal → better generalization. Combined with removing NSP and training longer on more data, this alone accounts for most of RoBERTa's gains.

RoBERTa results

Consistent improvements over BERT

Task (metric)	BERT-Large	RoBERTa	Improvement
SQuAD 2.0 (F1)	83.1	**89.4**	+6.3
MNLI (accuracy)	86.7	**90.2**	+3.5
SST-2 (accuracy)	94.9	**96.4**	+1.5
RACE (accuracy)	72.0	**83.2**	+11.2

Understanding the benchmarks

SQuAD 2.0: reading comprehension + unanswerable questions (human F1: 89.5 — RoBERTa essentially matches humans). **MNLI**: natural language inference across 10 genres (human: 92.0%, random: 33.3%). **SST-2**: binary sentiment classification (human: ~97%, random: 50%). **RACE**: multiple-choice reading comprehension from English exams (human: 92.2%, random: 25%). The RACE gain (+11.2 pts) is especially striking because it requires multi-sentence reasoning — exactly the capability that better training unlocks.

Key findings

- Dynamic masking consistently outperforms static masking
- More data + longer training = substantial gains
- Removing NSP improves downstream task performance
- Some tasks see enormous gains (RACE: +11.2 points!)

ALBERT: a lite BERT

Key idea: parameter sharing for efficiency

ALBERT (Lan et al., 2019) dramatically reduces BERT's parameter count through three innovations:

1. Factorized embedding parameters:

- BERT: vocabulary (30K) × hidden size (768) = 23M parameters
- ALBERT: vocabulary (30K) × embedding size (128) + embedding (128) × hidden (768) = 3.9M parameters
- **83% fewer embedding parameters**

2. Cross-layer parameter sharing:

- All 12 transformer layers share the *same* weights
- Like running the same layer 12 times (iterative refinement)
- **89% fewer transformer parameters**

3. Sentence Order Prediction (SOP):

- Replaces NSP with a harder task: are sentences A, B in the correct order?
- Forces the model to learn discourse coherence, not just topic matching

Factorized embedding parameters

Decomposing a large matrix into two small ones

BERT embeds each token directly into the hidden dimension $C = 768$. ALBERT introduces a bottleneck $E = 128 \ll C$:

$$\underbrace{V \times C}_{\text{BERT: 30K}\times\text{768}} \longrightarrow \underbrace{V \times E}_{\text{30K}\times\text{128}} \times \underbrace{E \times C}_{\text{128}\times\text{768}}$$

This factorization applies to the **token embeddings**, which dominate the parameter count. Positional ($512 \times E$) and segment ($2 \times E$) embeddings also use E, but they're tiny. All three are summed in E-space, then projected to C with a single linear layer.

Why this works

The vocabulary matrix is low-rank — tokens cluster into a much smaller subspace than $C = 768$. Embeddings only need to encode **token identity**; the projection layer handles the mapping to hidden space. Result: **83% fewer embedding parameters** (23M → 3.9M).

ALBERT parameter efficiency

Per-model parameter counts

Model	Layers	Hidden size	Parameters
BERT-base	12	768	110M
ALBERT-base	12	768	**12M**
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Factorized embedding in Python

Try it yourself!

```python
import torch.nn as nn

# BERT: direct embedding (30K vocab × 768 hidden = 23M params)
bert_embed = nn.Embedding(30000, 768)

# ALBERT: two-step embedding (30K × 128 + 128 × 768 = 3.9M params)
albert_embed = nn.Embedding(30000, 128)
albert_project = nn.Linear(128, 768)
# 83% fewer embedding parameters!
```

Cross-layer parameter sharing: BERT vs ALBERT

BERT layer structure

```python
class BERT:
    def __init__(self):
        # Each layer has unique parameters
        self.layers = [TransformerLayer() for _ in range(12)]
        # 12 × 7M params = 85M params in transformer layers

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)   # Different weights each time
        return x
```

Cross-layer parameter sharing: BERT vs ALBERT

ALBERT layer structure

```python
class ALBERT:
    def __init__(self):
        # Single shared layer
        self.shared_layer = TransformerLayer()
        # 1 × 7M params = 7M params (89% reduction!)

    def forward(self, x):
        for _ in range(12):
            x = self.shared_layer(x)  # Same weights reused
        return x
```

Trade-off

ALBERT has 89% fewer parameters but the **same compute cost** — it still performs 12 forward passes through the transformer layer. The savings are in memory, not speed.

DistilBERT: knowledge distillation

Key idea: train a small model to mimic a large model

Knowledge distillation (Sanh et al., 2019) compresses BERT into a smaller, faster model:

- **Teacher**: Full BERT-base (12 layers, frozen)
- **Student**: DistilBERT (6 layers, trainable)
- **Training signal**: Student learns to match the teacher's *soft probability distributions*, not just the hard labels

The teacher's "wrong" predictions contain useful information — e.g., predicting "cat" is more likely than "car" for a masked animal slot tells the student about semantic similarity.

Results

Metric	BERT-base	DistilBERT	Change
Parameters	110M	66M	40% smaller
Inference speed	1×	1.6×	60% faster
GLUE score	79.6	77.0	97% retained

What is GLUE?

GLUE (General Language Understanding Evaluation) is a benchmark suite of 9 tasks — including MNLI, SST-2, and others — that tests grammar, sentiment, similarity, and inference. The score is an average across all tasks; human baseline is ~87.

DistilBERT training in Python

Knowledge distillation training loop

```
teacher = BertModel.from_pretrained("bert-base")  # 12 layers, frozen
student = DistilBertModel(num_layers=6)            # 6 layers, trainable

for batch in training_data:
    # Teacher provides "soft targets" (probability distributions)
    with torch.no_grad():
        teacher_logits = teacher(batch)  # e.g., [0.7, 0.2, 0.1, ... ]

    # Student tries to match teacher's distribution
    student_logits = student(batch)

    # Distillation loss: KL divergence between soft distributions
    # Temperature T=2 softens the distribution (more informative)
    loss_distill = KL_divergence(
        softmax(student_logits / T),
        softmax(teacher_logits / T)
    )
    loss_mlm = masked_lm_loss(student_logits, labels)

    loss = 0.5 * loss_distill + 0.5 * loss_mlm
```

ELECTRA: efficient learning from *all* tokens

Key idea: learn from 100% of tokens, not just 15%

ELECTRA (Clark et al., 2020) uses a **generator-discriminator** setup:

1. A small **generator** (like a mini-BERT) fills in masked tokens with plausible replacements
2. A **discriminator** classifies *every* token as original or replaced
3. The discriminator provides a training signal for **all tokens** — not just the 15% that were masked

Why this is harder (and better) than MLM

BERT's MLM replaces tokens with `[MASK]` — an obvious tell that never appears in real text. The discriminator's task is harder: the generator produces *plausible* substitutions ("ate" for "cooked"), so the discriminator must understand the full context deeply enough to detect subtle semantic mismatches. This is closer to how humans process language — we don't spot blank slots, we notice when something *doesn't quite fit*.

ELECTRA in Python

Complete example — paste into Colab and run!

```
import torch
from transformers import AutoTokenizer, AutoModelForMaskedLM, ElectraForPreTraining

tokenizer = AutoTokenizer.from_pretrained("google/electra-small-generator")
generator = AutoModelForMaskedLM.from_pretrained("google/electra-small-generator")
discriminator = ElectraForPreTraining.from_pretrained("google/electra-small-discriminator")

# Step 1: Mask a token and let the generator fill it in
original = "The chef cooked a delicious meal"
masked = "The chef [MASK] a delicious meal"
inputs = tokenizer(masked, return_tensors="pt")

with torch.no_grad():
    gen_logits = generator(**inputs).logits

mask_idx = (inputs.input_ids == tokenizer.mask_token_id).nonzero(as_tuple=True)[1]
predicted_id = gen_logits[0, mask_idx].argmax(dim=-1)
replacement = tokenizer.decode(predicted_id)
print(f"Generator filled [MASK] → '{replacement}'")  # e.g., "prepared"

# Step 2: Discriminator classifies EVERY token as original or replaced
fake_ids = inputs.input_ids.clone()
fake_ids[0, mask_idx] = predicted_id
with torch.no_grad():
    disc_logits = discriminator(fake_ids).logits

```

ELECTRA efficiency

Learning from every token

ELECTRA learns from **100% of tokens** (every position gets a real/replaced label), compared to BERT's 15%. This 6.7× increase in training signal means ELECTRA reaches BERT-level performance with **4× less compute**.

When to use ELECTRA

ELECTRA is ideal when you have limited compute budget:

- ELECTRA-Small outperforms BERT-Small
- ELECTRA-Base is competitive with BERT-Large
- With the same compute budget, ELECTRA consistently wins

Variant comparison summary

Choosing the right BERT variant

Model	Key innovation	Best for
BERT	MLM + NSP	Baseline, well-understood
RoBERTa	Better training recipe	Maximum quality
ALBERT	Parameter sharing	Memory-constrained deployment
DistilBERT	Knowledge distillation	Speed-critical production
ELECTRA	Replaced token detection	Limited training budget

General guidelines

- **Best quality:** RoBERTa-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT
- **Limited training budget:** ELECTRA
- **Good default:** RoBERTa-Base or BERT-Base

Other notable BERT variants

The BERT family keeps growing

DeBERTa (Microsoft, 2020): Disentangled attention separates content and position representations. Enhanced mask decoder. State-of-the-art on SuperGLUE.

SpanBERT (Facebook, 2019): Masks random contiguous *spans* instead of individual tokens. Span boundary objective. Better for extractive tasks (QA, coreference).

ERNIE (Baidu, 2019): Entity-level and phrase-level masking. Knowledge-enhanced pre-training. Strong on Chinese NLP tasks.

BART (Facebook, 2019): Encoder-decoder architecture (not encoder-only). Denoising autoencoder with various corruption strategies. Excellent for generation tasks.

Model selection guide

How to choose the right model for your task

Step 1: What are your constraints?

- Need maximum quality? → **RoBERTa-Large**
- Need fast inference? → **DistilBERT**
- Need small memory footprint? → **ALBERT**
- Limited training compute? → **ELECTRA**
- Need text generation? → Consider **BART** or **GPT** instead

Step 2: Consider your domain

- Biomedical text? → **BioBERT**, **PubMedBERT**
- Scientific text? → **SciBERT**
- Legal text? → **LegalBERT**
- Multilingual? → **mBERT**, **XLM-RoBERTa**

Step 3: Start simple, iterate

- Begin with `bert-base-uncased` as a baseline

Using different variants with HuggingFace

Easy model switching with AutoModel

```python
from transformers import AutoModel, AutoTokenizer

# All variants share the same API — just change the model name!

# BERT
model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# RoBERTa
model = AutoModel.from_pretrained("roberta-base")
tokenizer = AutoTokenizer.from_pretrained("roberta-base")

# ALBERT
```

continued...

Using different variants with HuggingFace

Easy model switching with AutoModel

```
model = AutoModel.from_pretrained("albert-base-v2")
tokenizer = AutoTokenizer.from_pretrained("albert-base-v2")

# DistilBERT
model = AutoModel.from_pretrained("distilbert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# ELECTRA
model = AutoModel.from_pretrained("google/electra-base-discriminator")
tokenizer = AutoTokenizer.from_pretrained("google/electra-base-discriminator")
```

...continued

Benchmarking variants

Comparing models on sentiment analysis

```python
import time
from transformers import pipeline

models = {
    "bert-base": "textattack/bert-base-uncased-SST-2",
    "distilbert": "distilbert-base-uncased-finetuned-sst-2-english",
    "albert": "textattack/albert-base-v2-SST-2",
}
test_texts = ["This movie was fantastic!", "I hated every minute."] * 100

for name, model_id in models.items():
    pipe = pipeline("sentiment-analysis", model=model_id)
    start = time.time()
    results = pipe(test_texts)
```

Architecture evolution

From BERT to ModernBERT: 6 years of progress

Model	Year	Key innovation	Quality vs BERT
BERT	2018	MLM + NSP	Baseline
RoBERTa	2019	Better training recipe	+3–11 pts
ALBERT	2019	Parameter sharing	89% fewer params
DistilBERT	2019	Knowledge distillation	97% quality, 60% faster
ELECTRA	2020	Learn from all tokens	4× less compute
DeBERTa	2020	Disentangled attention	SOTA on SuperGLUE
ModernBERT	2024	Modern training + RoPE + Flash Attention	SOTA on GLUE, retrieval

ModernBERT deep dive

Warner et al. (Dec 2024): bringing modern LLM techniques to encoders

ModernBERT applies 6 years of decoder innovations to the encoder architecture:

Innovation	BERT (2018)	ModernBERT (2024)
Position encoding	Learned absolute (512 max)	**RoPE** (8,192 tokens)
Attention	Full quadratic	**Flash Attention** + alternating global/local
Padding	Processes pad tokens	**Unpadding** (only real tokens)
Training data	3.3B words	**2 trillion tokens** (600× more)
Code understanding	None	Trained on code corpora

Results

SOTA on GLUE, retrieval (MTEB), and code understanding benchmarks. Available as `answerdotai/ModernBERT-base` and `answerdotai/ModernBERT-large` on HuggingFace. Proves that encoder architecture still has room to grow when given modern training techniques.

Gemma Encoder and the encoder renaissance

Google bets on encoders again (2025)

Gemma Encoder (2025): Google's first encoder-only model since BERT, built by repurposing Gemma decoder weights for bidirectional encoding. Competitive with ModernBERT on sentence embedding tasks.

Why this matters

If Google — a company betting heavily on decoder-only models (Gemini) — still releases an encoder model, it signals that encoders serve a purpose decoders can't efficiently fill. The encoder isn't dead; it's being **modernized**.

Production deployment patterns

Getting encoders from prototype to production

Optimization	Speedup	Memory savings	Quality loss
ONNX Runtime	2–5×	Moderate	None
TensorRT (NVIDIA)	5–10×	Moderate	None
INT8 quantization	2–4×	**4× smaller**	<1%
Distillation (→ DistilBERT)	1.6×	40% smaller	~3%
Full pipeline (distill → quantize)	10–20×	**8× smaller**	~4%

The cost argument

DistilBERT + INT8 quantization handles classification at ~$0.001/M tokens and ~2ms per request. GPT-4 costs ~$30/M tokens at ~500ms per request. For high-volume single-task workloads, optimized encoders are **30,000× cheaper** and **250× faster**.

Is the encoder dead?

Discussion: GPT-4 can classify text via prompting. Do we still need encoders?

The case for "yes, encoders are obsolete":

- Decoder-only models (GPT-4, Claude) can do classification, NER, QA via prompting
- One model for all tasks vs. fine-tuning separate models
- In-context learning eliminates the need for task-specific architectures

The case for "no, encoders still matter":

- ModernBERT (2024) achieves SOTA on retrieval and classification — faster and cheaper than any decoder
- Gemma Encoder (2025) proves Google still sees value in the architecture
- Encoders are 10–100× cheaper to run than decoder models for classification tasks
- Most production search and retrieval systems still use encoders (Sentence-BERT, E5, NV-Embed)

The real answer: It depends on your constraints. Encoders win on cost and latency

Discussion

Questions to consider

1. **Training vs architecture:** RoBERTa shows that training matters enormously. How much of BERT's "limitations" were really just undertrained models?
2. **The distillation paradox:** Why does a student model learn *better* from soft probability distributions than from hard labels? What "dark knowledge" is in the teacher's mistakes?
3. **Encoders in 2026:** Google, Hugging Face, and others are *still* releasing encoder models. If decoders can do everything, why? What does this tell us about the efficiency-flexibility tradeoff?
4. **Parameter sharing (ALBERT):** All 12 layers share the same weights and it still works. What does this imply about what transformer layers actually learn?

Further reading

Further reading

Liu et al. (2019, *arXiv*) "RoBERTa: A Robustly Optimized BERT Pretraining Approach" — Better training recipe, same architecture.

Lan et al. (2019, *ICLR*) "ALBERT: A Lite BERT" — 89% parameter reduction via sharing.

Sanh et al. (2019, *NeurIPS Workshop*) "DistilBERT" — Knowledge distillation for 60% speedup.

Clark et al. (2020, *ICLR*) "ELECTRA" — Learn from all tokens, not just masked ones.

He et al. (2020, *ICLR*) "DeBERTa" — Disentangled attention, SOTA on SuperGLUE.

Warner et al. (2024, *arXiv*) "ModernBERT" — Modern encoder with RoPE + Flash Attention.

Google (2025, *arXiv*) "Gemma Encoder" — Encoder-only Gemma for sentence embeddings.

Questions?

Email

Discord

Office hours

Up next...

Applications of encoder models: industry, neuroscience, and the "understanding" debate